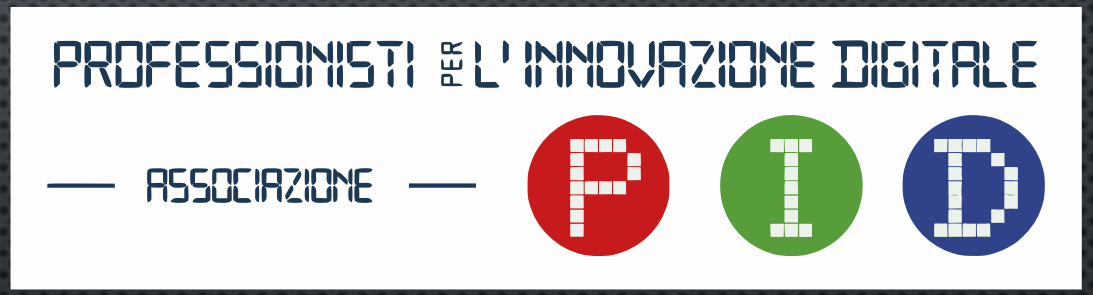
INFORMATIONAL MEETING ON HARDWARE DESCRIPTION LANGUAGES (HDL)

SPEAKERS:

GIULIANO CARDINALI

EMANUEL CONTI



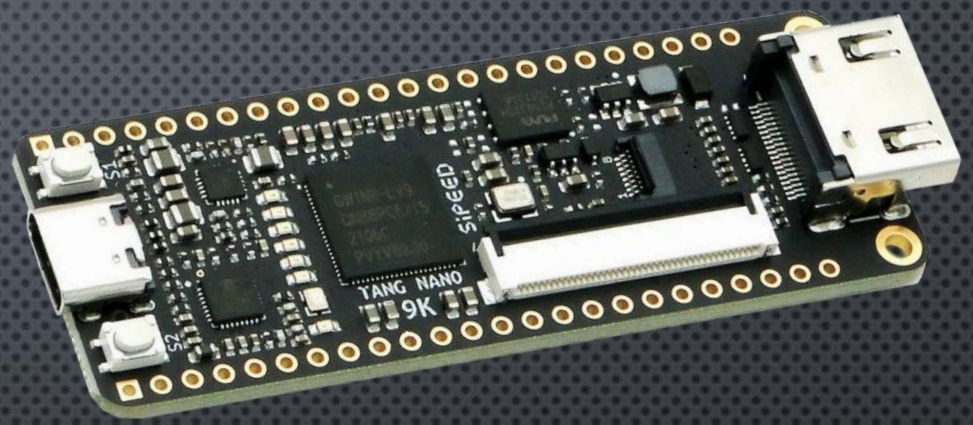
ORGANIZED BY:

https://www.pidrsm.sm
https://www.facebook.com/professionistiperinnovazionedigitale

HDL IN PRACTICE: DESIGN, SYNTHESIS AND SIMULATION

PRESENTATION OF THE DEVELOPMENT KIT

For this presentation we chose an easily available, low-cost development kit.



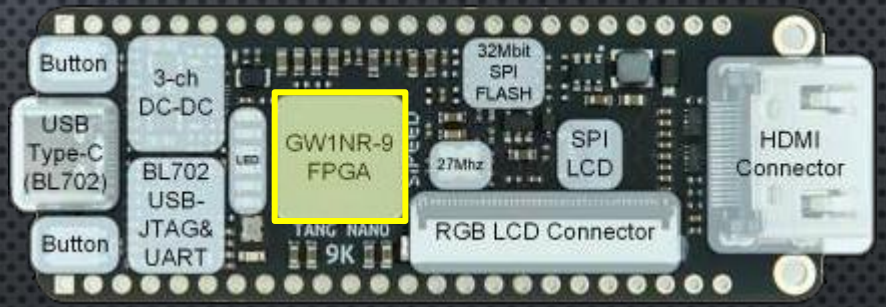
Specifically, we will use the SiPeed Tang Nano 9K kit.

Tang Nano 9K uses a non-volatile Flash-based FPGA from GOWIN, belonging to the LittleBee® family, with integrated RAM (GW1NR). It offers just under 9000 4-input LUTs/CLBs, DSP functions and several modules, including two PLLs.

TANG NANO 9K BY SIPEED (GOWIN GW1NR FPGA)

Technical specifications

Logic Units (LUT4)	8640
Crystal oscillator	27 MHz
SPI FLASH (bits)	32 M SPI flash (Onboard)
Registers (Flip Flops)	6480
Shadow SRAM SSRAM (bits)	17280
Block SRAM B-SRAM (bits)	468 K (26 x 18K)
Number of B-SRAM	26
User flash (bits)	608 K
SDR SDRAM (bits)	64 M
18 x 18 Multiplier	20
Number of PLL	2
Display interface	HDMI interface, SPI screen interface and RGB screen interface
Debugger	Onboard BL702 chip with USB-JTAG and USB-UART
I/O	4-24 mA, Bus Keeper, pull-up/pull-down resistors, Open Drain output options for each I/O
Connector	TF card slot, 2x24 P 2.54 mm Header pads
Button	2 programmable buttons for users
LED	Onboard 6 programmable LEDs



DEVELOPMENT SYSTEMS (IDE)

There are two development systems (toolchains) that can be used with the TangNano 9K:

- The manufacturer's IDE/Toolchain: GoWin FPGA Designer. It supports all GoWin FPGA families and offers excellent integration of all the available modules (hard/soft IP). It has no built-in simulator but can use an external simulator or a Cloud resource accessible from the IDE.
- VS Code + Lushay Code plugin by Lushay Labs:

  https://learn.lushaylabs.com/vscode-extension/

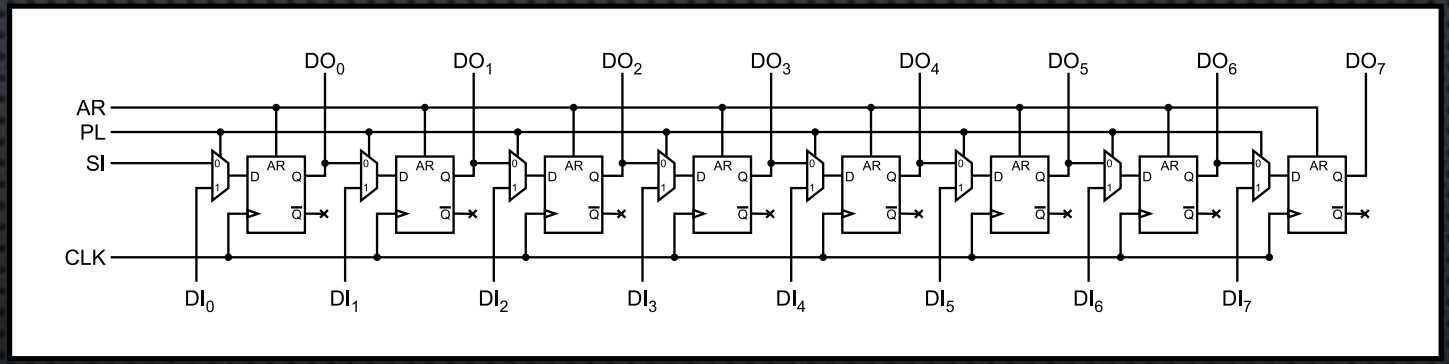
  Lushay Code uses the OSS-Cad-Suite toolchain (Yosys, Apicula-nextpnr, openFPGALoader, iverilog, etc.).

  https://github.com/YosysHQ/oss-cad-suite-build
- It does not cover all the features (hard IP) specific to GoWin's technology and supports only a limited number of devices.

HDL IN PRACTICE: DESIGN

SHIFT REGISTERS

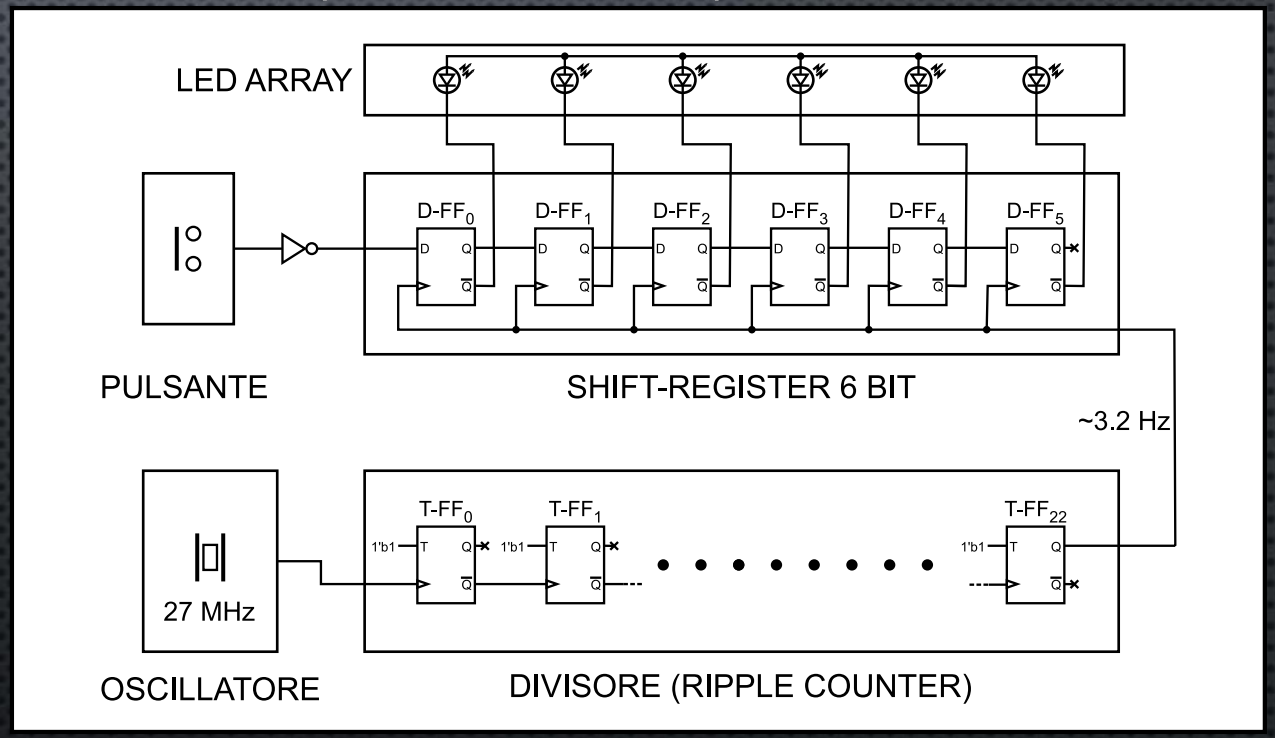
- Shift registers can be used to convert data from parallel to serial (serializer) or from serial to parallel (deserializer). They consist of D-type Flip-Flops in cascade (the output of one Flip-Flop feeds the next). The clock inputs of the Flip-Flops are all tied together, as is any synchronous or asynchronous reset.
- If the states of the Flip-Flops can be assigned simultaneously in a single clock cycle (Parallel Load), a Shift Register can be used as a serializer (PISO, Parallel In Serial Out) by taking the output of the last Flip-Flop.
- A Shift Register that exposes all the outputs of its Flip-Flops can be used as a deserializer (SIPO, Serial In Parallel Out).



A PRACTICAL EXAMPLE WITH THE DEVELOPMENT KIT (1)

To show how the IDE is used, making use of the resources on the development kit, we will present a small project that includes a synchronous sequential network.

It will be a 6-bit shift register whose serial input is driven by one of the two buttons on the development board. The parallel output of the Shift Register will be shown on the Tang Nano 9K's LEDs.

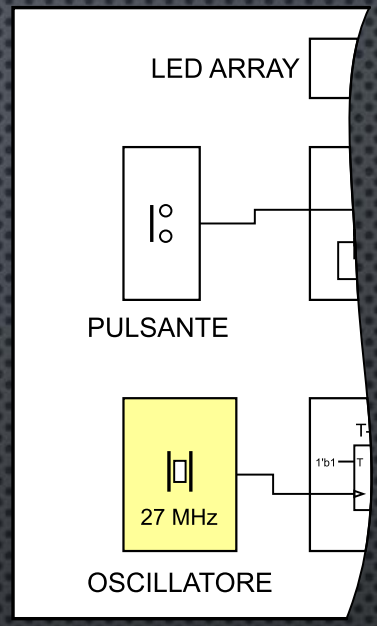


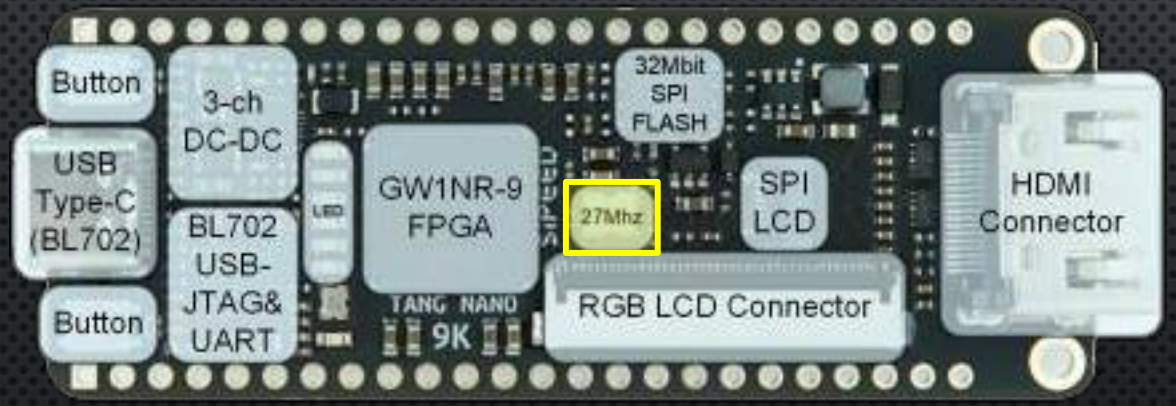
A PRACTICAL EXAMPLE WITH THE DEVELOPMENT KIT (2)

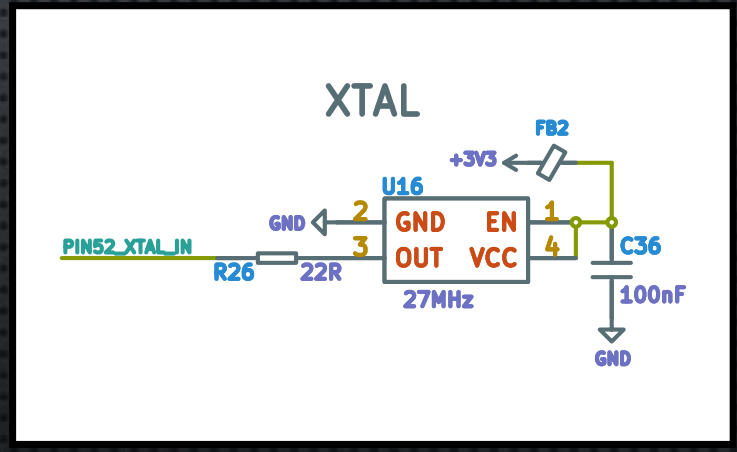
The TangNano 9K has an XO (Crystal Oscillator) with good frequency stability, capable of generating a 27 MHz signal.

XO means an oscillator module controlled by a quartz crystal.
Such a device gets its frequency stability from that intrinsic to the crystal, which is modestly sensitive to temperature variations. This characteristic is generally specified in tens of parts per million (ppm). The initial accuracy at room temperature (+25 °C) is mainly given by the calibration of the crystal by the module's manufacturer.

We will use this time reference, suitably processed, to provide a cadence (a clock) to the logic network we will define.







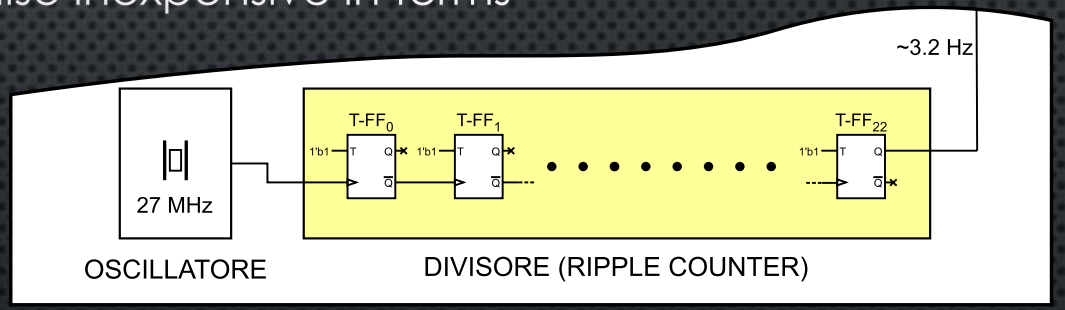
A PRACTICAL EXAMPLE WITH THE DEVELOPMENT KIT (3)

The 27 MHz output is too fast for our experiment, since we would like to be able to appreciate the effect of the Shift Register shifting directly on the LEDs of the Tang Nano 9K.

To bring the speed to a suitable value (about 3.2 Hz) we must "divide" the XO frequency by 8,388,608: this is not a random value, but corresponds to the number obtained by calculating 2^{23}.

Building a synchronous counter with 8,388,608 states to divide the frequency requires the combinational network that computes the next value to be of non-negligible complexity and considerable size. Moreover, we are talking about a logic network (an adder) with many levels (up to 23).

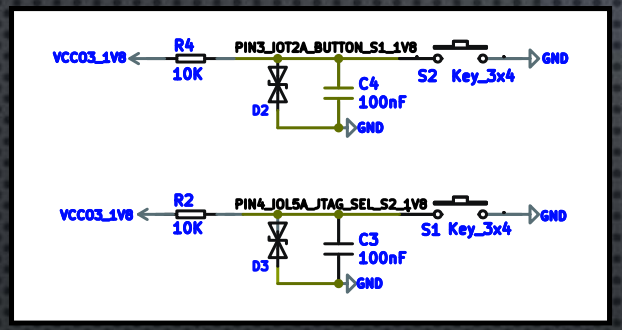
Since we are not interested in the whole state of the counter but only in the state of the most significant bit, and since in this application we have a single clock, we can opt for a so-called "Ripple Counter". Besides being very fast, it is also inexpensive in terms of resources. On the downside, its outputs do not switch synchronously with one another (not simultaneously), but for our application we know this is not a problem.

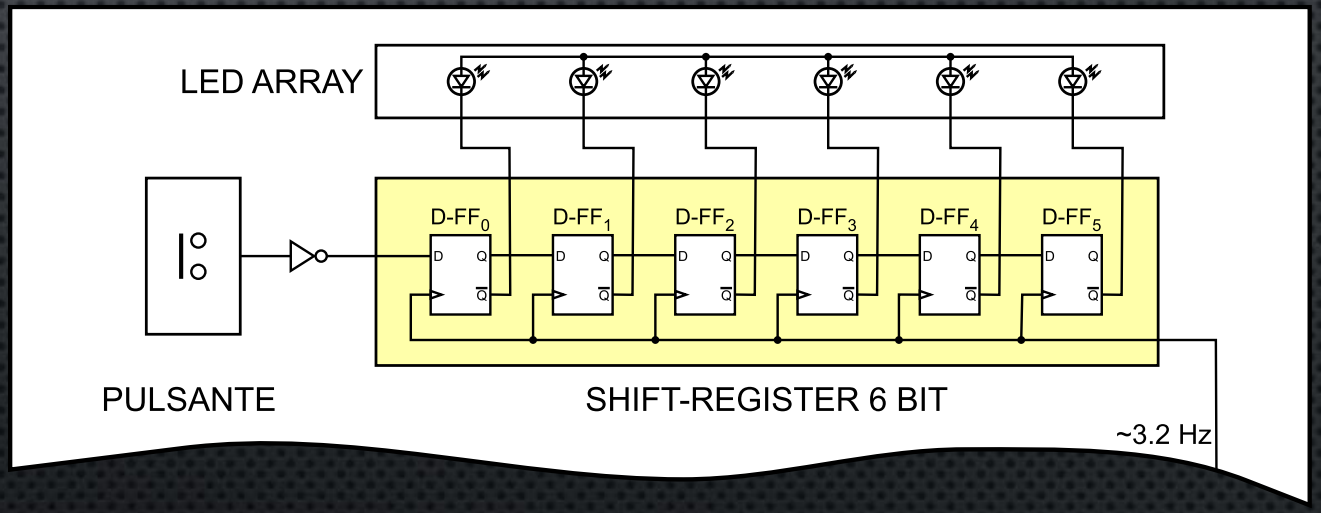


The classic structure has the inverted output of a T-type Flip-Flop (T stands for "toggle", i.e. it inverts its outputs when the T input is 1) act as the clock source for the next flip-flop. After several successive divisions by two (up to the figure of 8,388,608) of the initial 27 MHz frequency, we finally obtain a cadence of 3.218… Hz.

A PRACTICAL EXAMPLE WITH THE DEVELOPMENT KIT (4/1)

The clock coming out of the Ripple Counter is connected to the clock input of a six-bit Shift Register. For simplicity, the figure omits the asynchronous reset connected in parallel to all the Flip-Flops. This reset comes from one of the two buttons of the TangNano 9K, suitably inverted because it is normally active low (zero when pressed).



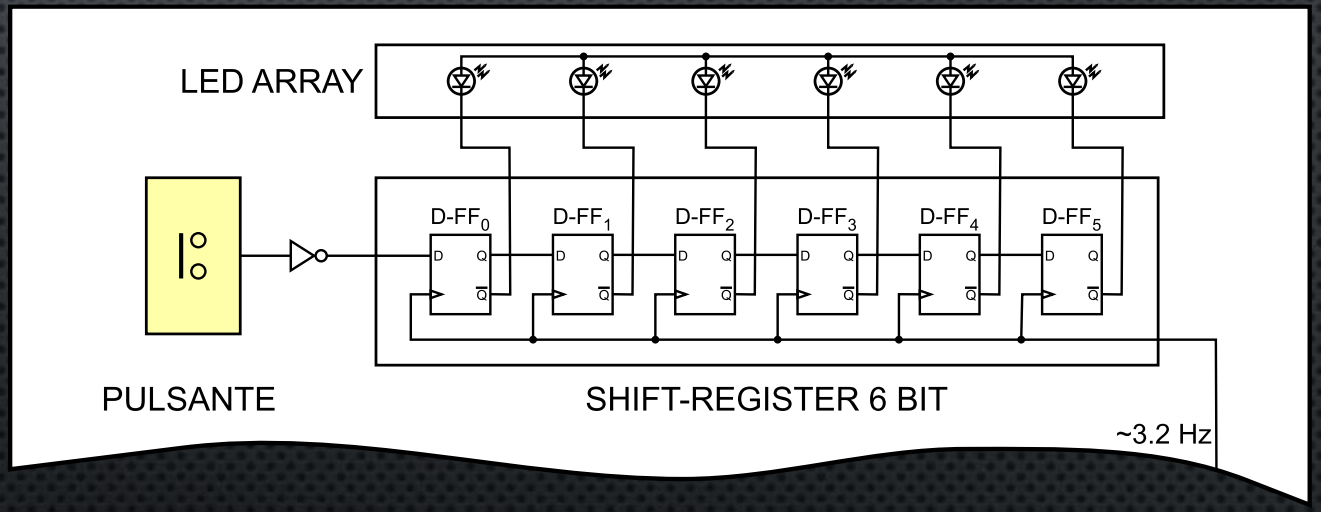


A PRACTICAL EXAMPLE WITH THE DEVELOPMENT KIT (4/2)

The clock coming out of the Ripple Counter is connected to the clock input of a six-bit Shift Register. For simplicity, the figure omits the asynchronous reset connected in parallel to all the Flip-Flops. This reset comes from one of the two buttons of the TangNano 9K, suitably inverted because it is normally active low (zero when pressed).

The serial input of the shift register is connected to the second button, on the board, which is also inverted.

On every rising edge of the clock, the Shift Register copies the state of the button connected to its input and transfers it to the outputs.

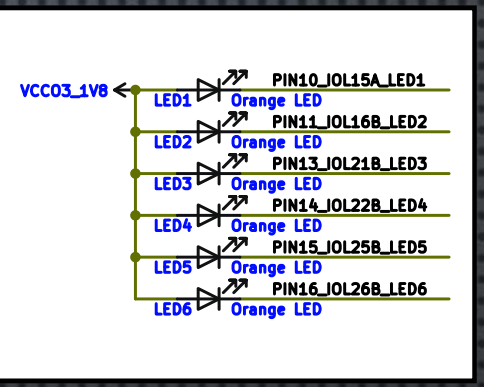


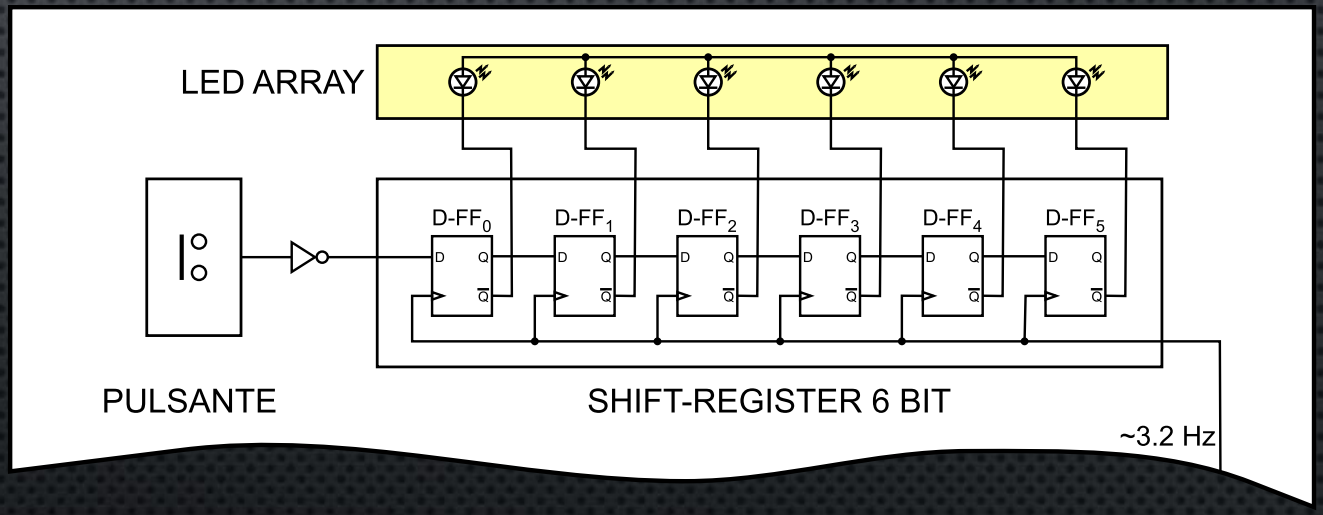
A PRACTICAL EXAMPLE WITH THE DEVELOPMENT KIT (5)

The LEDs on the board are connected to the outputs of the six-bit Shift Register.

Note that the signals that turn on the LEDs are taken from the inverted outputs of the Shift Register's Flip-Flops. This is necessary because, on the Tang Nano 9K board, the LEDs are all "tied" to the positive supply on the anode side (the LEDs also incorporate a current-limiting resistor).

This means that to turn an LED on, its cathode must be brought to ground, which is equivalent to emitting a zero level on the output.





HDL IN PRACTICE: A DATA TRANSMISSION EXAMPLE

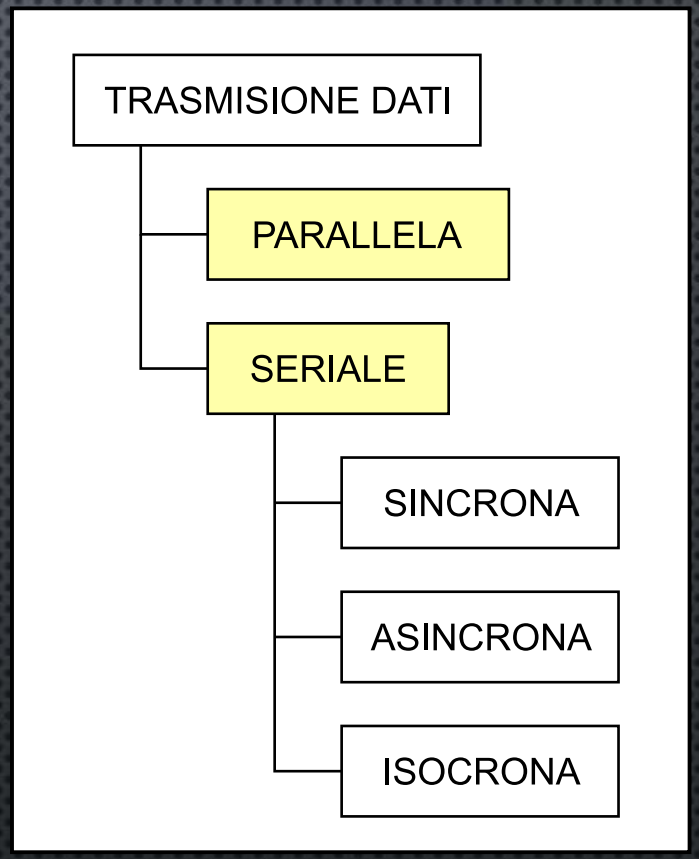
DATA TRANSMISSION

Parallel data transmission:

It is generally synchronous, with the cadence dictated by a clock. It uses *n* wires to send the bits of a word all at the same time. Each bit has its own wire and all the bits are transmitted in a single clock beat. Additional connections may be present to manage the flow (the so-called handshake) between transmitter and receiver.

Serial data transmission:

In serial transmission one bit follows the other in time. Therefore only one wire is needed to transmit the data. Additional connections may be required to manage the flow (handshake), but are not always present. Serial data transmission can be divided into three broad categories: synchronous, asynchronous and isochronous.



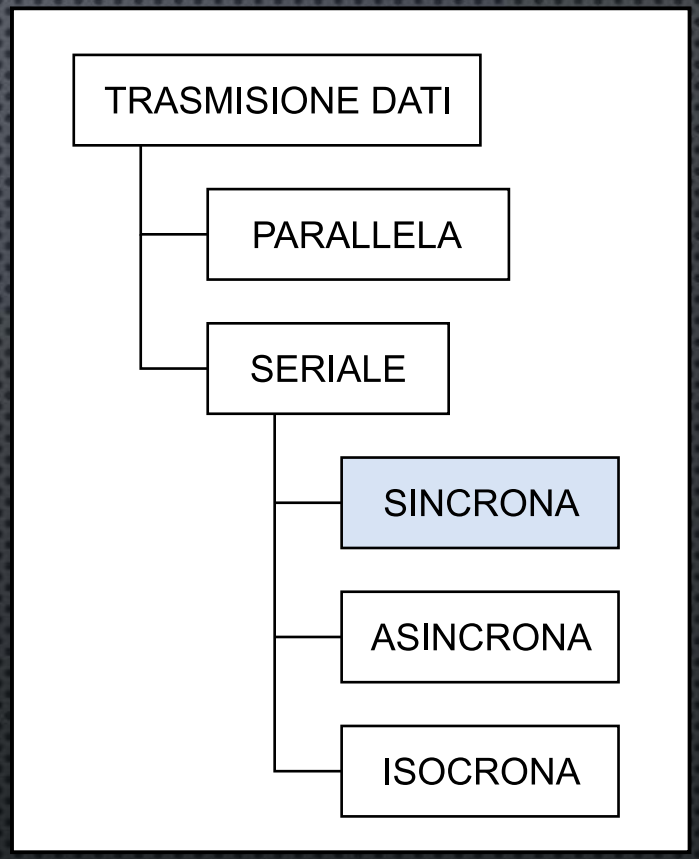
MODES OF SERIAL DATA TRANSMISSION (1)

Synchronous transmission:

In synchronous transmission, the bit stream is combined into longer structures that can contain several words (for example eight-bit words, i.e. bytes). Each word is sent with no gaps between one and the next. The receiver must decode the bit stream and recognize the beginning and the end of the words. That is, since the data are transmitted as an uninterrupted sequence of 1s and 0s, it is the receiver that reassembles the information starting from that string of bits.

It is called synchronous transmission because the information is always accompanied by a precise cadence (clock), which can arrive on an additional connection or be embedded in the data stream with the help of special encodings. Hardware interfaces such as SPI or I2C have a dedicated clock on a separate wire. Protocols such as Manchester, although requiring twice the bandwidth, can embed the cadence directly in the signal, thus requiring only a single connection.

Synchronous serial transmission makes it possible to achieve transfer rates that are usually very high.



MODES OF SERIAL DATA TRANSMISSION (2)

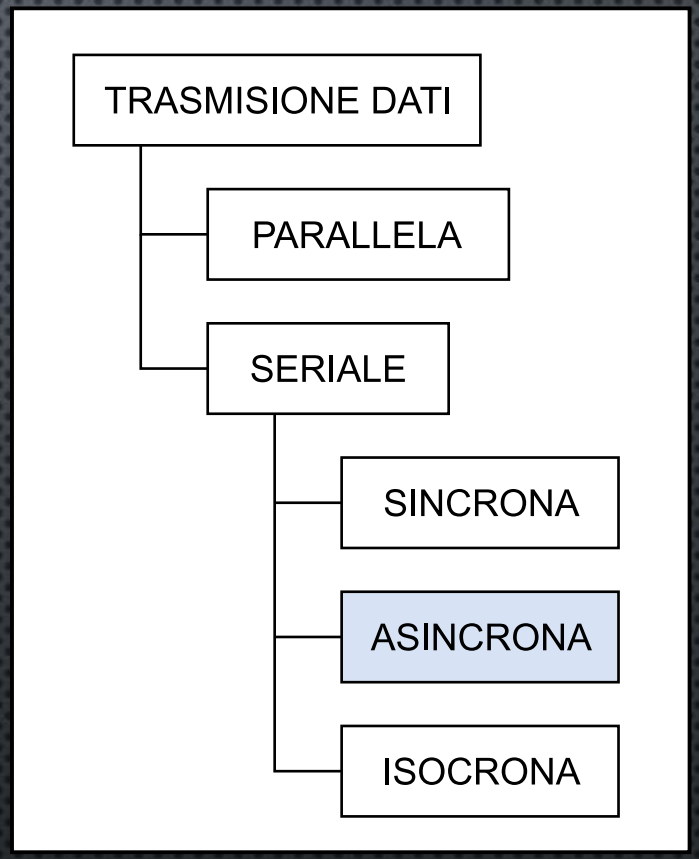
Asynchronous transmission:

Asynchronous transmission is so called because the information is transmitted according to an agreed protocol without a precise time reference (clock).

Usually the bits in sequence are grouped to form words (bytes). Each word is sent as a fixed-length group of bits, and each group is handled independently and sent only when it is fully available. Without synchronization, the receiver cannot rely on precise instants of time to predict when the next group will arrive.

To warn the receiver of the arrival of a new group, an extra bit is added at the beginning of each word (byte). This bit, usually a 0, is called the start bit. To let the receiver know that the word has ended, one or more bits set to 1 are added at the end of the word itself. These bits are called stop bits. With this method, the size of each word is increased by at least two bits, namely the start bit and the stop bits. Furthermore, the transmission of a word may be followed by an interval of variable duration (gap).

By its nature, asynchronous transmission limits the speed to a few hundred KB per second.



MODES OF SERIAL DATA TRANSMISSION (3)

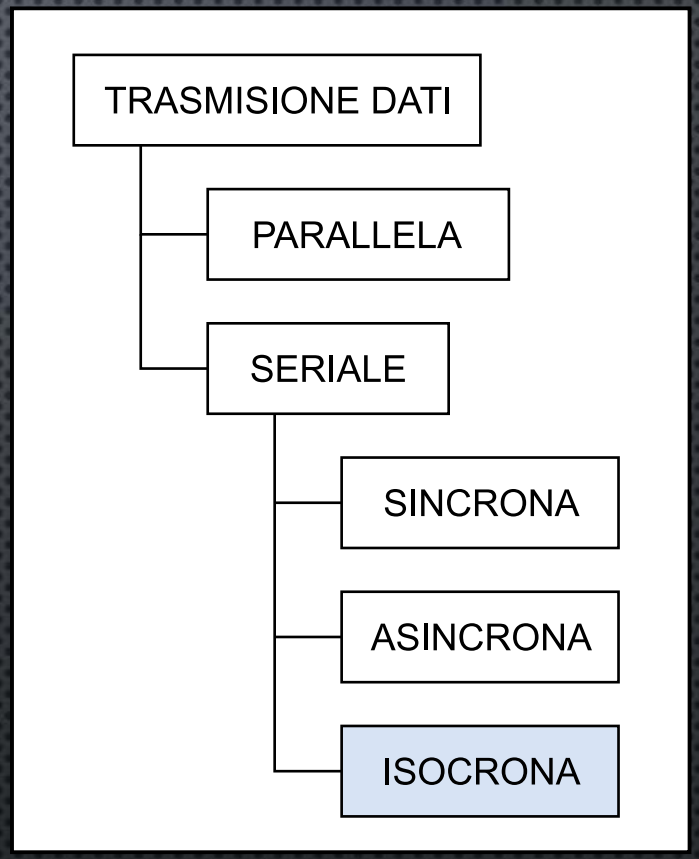
Isochronous transmission:

In isochronous transmission it is important that the information is received shortly after it was transmitted (low latency), and that the information is carried over channels that support medium-to-high base rates; in general the channel speed must be several times higher than that of the actual information to be transmitted.

Typical applications of isochronous transmission are real-time communications, such as audio, video, data from sensors that generate them at a regular and precise cadence, or the transport of data coming from edge computing.

As mentioned, the bandwidth of the transport channel must be *n* times (usually 4 times or more) greater than the bandwidth required by the information itself. The information can also be temporarily accumulated (buffering) as long as this does not cause excessive latency. The data are then transferred to the receiver at the highest possible speed.

After reception and decoding, the cadence of the information can be reconstructed as it originally was, if necessary. Indeed, in multimedia applications, for reasons of intelligibility or interpretation, it is important that the stream leaves the receiver at the same speed at which it entered the transmitter, albeit with some acceptable delay.



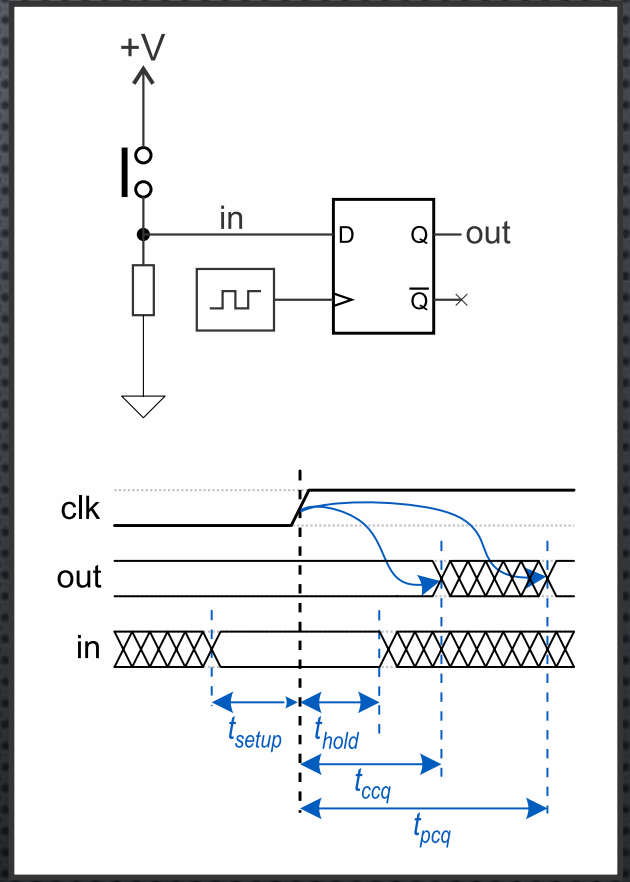
HDL IN PRACTICE: RELIABILITY OF LOGIC NETWORKS

EXTERNAL ASYNCHRONOUS EVENTS (1)

Consider the drawing alongside: a circuit with an input coming from a button connected to the D input of a Flip-Flop. When the button is not pressed, the flip-flop receives a stable value at its input. If the button is pressed, however, the flip-flop's input changes at a random time with respect to the clock.

Someone might press the button just as the clock rises. This can lead to a phenomenon called "metastability", in which the flip-flop captures at its input a value halfway between 0 and 1, which can take an unbounded amount of time to resolve into a stable logic value at the output.

When the clock rises and D is stable, the output can start to change after the clock-to-Q (so-called contamination) delay, t_{ccq}, and must definitively settle on the final value within the clock-to-Q propagation delay, t_{pcq}.

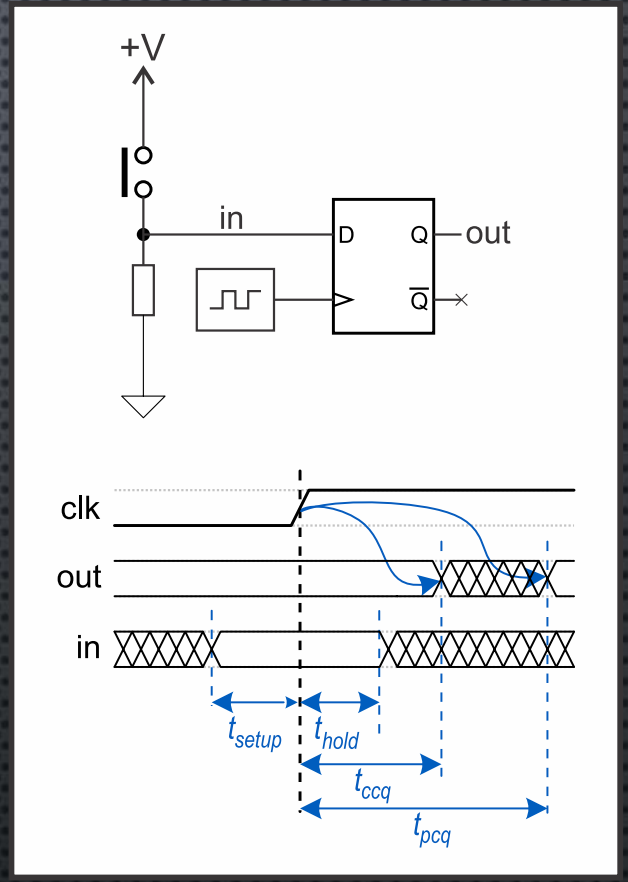


EXTERNAL ASYNCHRONOUS EVENTS (2)

The flip-flop has two fundamental parameters, t_{setup} and t_{hold}: these are two times for which the input is required to be stable with respect to the clock edge the flip-flop operates on. The sum of the t_{setup} and t_{hold} times is called the aperture time.

Violating either of these two times leads to the metastable condition at the flip-flop's output. When it becomes metastable there is no way to know with reasonable precision how long it will take to leave that state and what final value Q will take.

This condition of data being asynchronous with respect to the clock is found whenever there is no correlation between D and the clock: external inputs coming from contacts, sensors or signals from other equipment are generally asynchronous and therefore a potential cause of metastability.

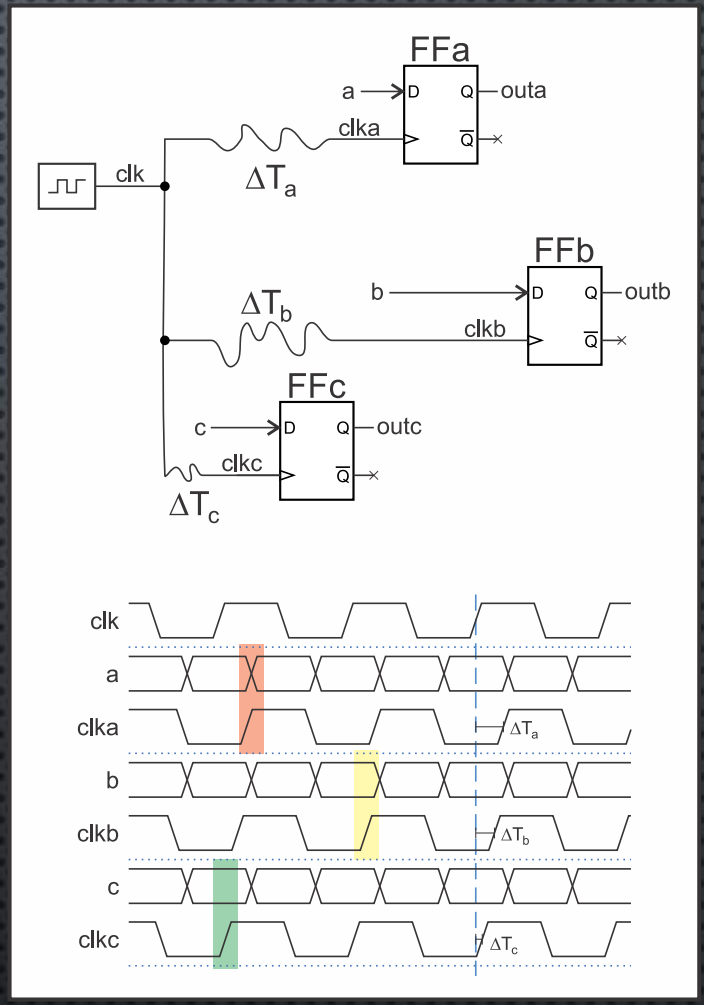


CLOCK SKEW

In real systems, the clock may not reach all the flip-flops at the same time. This variation in time is called "clock skew". The wires that carry the clock from its source to the registers, or noise on the clock itself, can produce different delays for each wire. Components inserted in the clock path (to obtain so-called clock gating) can also cause clock skew.

Even if on paper we have made sure that a clock domain is synchronized with all the signals related to it, the difference in clock propagation times at the inputs of the various flip-flops of a synchronous network can create metastability conditions. This also applies to the delay of the data relative to the clock.

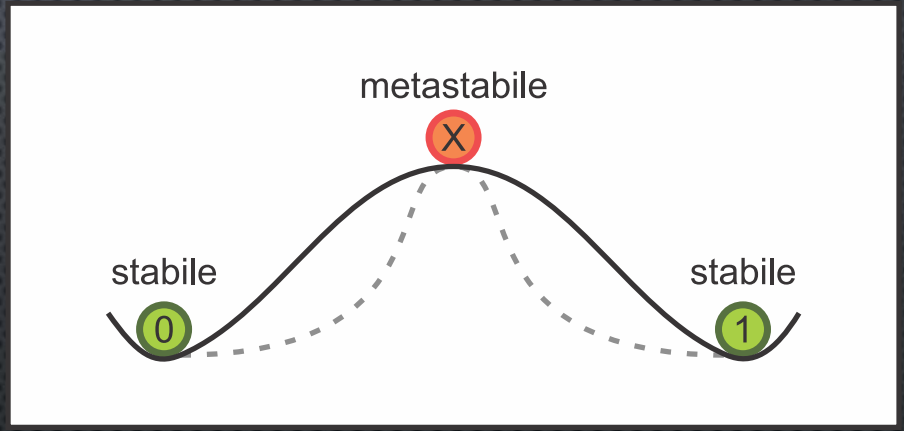
In modern programmable logic devices there are priority paths for clock signals, so that distribution to the various flip-flops and devices is highly uniform. These signals have a structure that also gives them a high fan-out. The fan-out of a digital output determines the number of inputs that can be connected to it (the load) without the signal degrading excessively.



METASTABLE CONDITION

In real situations, when a flip-flop samples an input that changes during its aperture time, the Q output can momentarily take a voltage between 0 and VDD (logic 1) lying in the "forbidden zone". This is the so-called metastable condition. Eventually, the flip-flop will resolve the output into a stable state (0 or 1). However, the resolution time needed to reach the stable state can be infinite or at any rate undetermined.

The metastable state of a flip-flop is analogous to that of a ball on the top of a hill between two valleys, as shown alongside.



The two valleys are the stable states, because a ball in the valley will stay there until it is disturbed. The top of the hill is called the metastable zone because the ball could stay there if subjected to a perfect balance of forces. But sooner or later, perhaps because of a slight breeze, the ball will roll to one side or the other. The time needed for this change to occur depends on how balanced the forces were, how narrow the hilltop is, and on disturbing factors (noise).

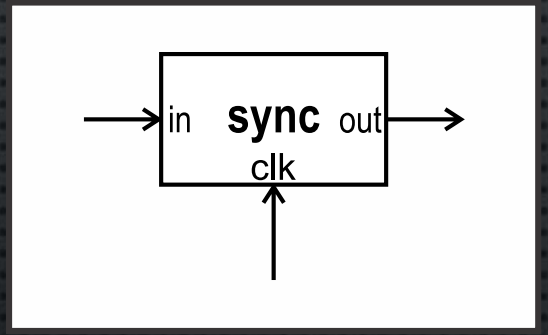
Every bistable device always has a metastable state between its two stable states.

SYNCHRONIZERS (1)

Consider a circuit with an input coming from an asynchronous source that changes, sometimes violating the t_{setup} and t_{hold} times and thus creating, if handled carelessly, metastability situations, i.e. causing irregular system failures that are extremely difficult to trace and fix. The goal of a digital system designer should be to ensure that, given asynchronous inputs, the probability of encountering a metastable condition is sufficiently small.

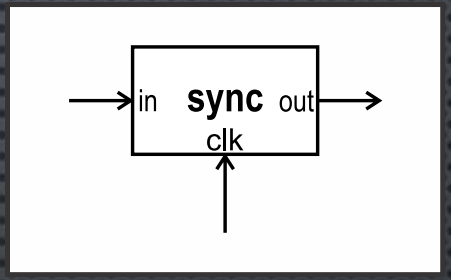
The concept of a sufficiently small probability depends on the application context: for a personal computer perhaps one error every 10 years is acceptable, because you just restart the PC if it freezes. For a medical device or the control of a ballistic missile, at most one failure in the entire lifetime of the Universe (10^{10}years) is better.

The solution for such asynchronous inputs is to pass them through synchronizers, which correlate the asynchronous input signal with the digital system's clock and have a very small (but always greater than zero) probability of producing an invalid logic value.



SYNCHRONIZERS (2)

A synchronizer, shown alongside, is a device that receives an asynchronous input (in) and a clock (clk). It produces an output (out) that, within a limited period of time, must have the right, valid logic level with an extremely high probability.

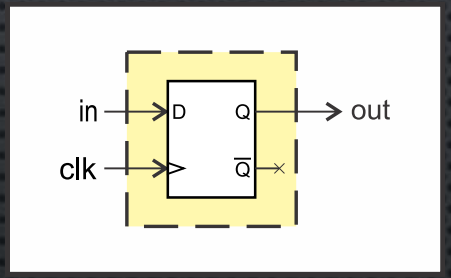


We say that a synchronizer fails if Q becomes metastable. A synchronizer is usually built with a high-quality D-type flip-flop.

If the D input is stable during the aperture, Q should take the same value as D. If D changes during the aperture, Q can take the value 1 or 0, but it should not be frequently metastable, or if it becomes so it should not remain in that condition for long.

System reliability is usually measured in MTBF (Mean Time Between Failures), which is the mean time between failures of the system itself. It is calculated as the reciprocal of the probability that the system fails in a given second:

$$MTBF = \frac{1}{P(Failures)/sec}$$



A typical equation for calculating the MTBF is:

$$MTBF = \frac{e^{S/\tau}}{T_w F_c F_D}$$

Where T_w is the sum of t_{setup} and t_{hold}, F_c is the clock frequency, F_D is the number of transitions per second at the D input, S is the time estimated for the metastable event to resolve (usually one clock cycle) and τ is a parameter related to the device technology (typically a gate delay).

HDL IN PRACTICE: THE "MANCHESTER" ENCODING

THE MANCHESTER ENCODING

Manchester encoding (named after the university where it was developed) is a binary encoding (or, if you like, a modulation) with phase shift (2-BPSK).

This encoding guarantees frequent potential transitions on the transmission line, which are directly proportional to the clock frequency, thus making it easier to recover (synchronize) the latter at the receiver. The Manchester code always has a transition in the middle of each bit.

The signal is usually stripped of its DC component, which is not relevant for interpreting the data. Normally, from the electrical point of view, differential transmission is chosen.

The transmission line needs a bandwidth at least double that of the information.

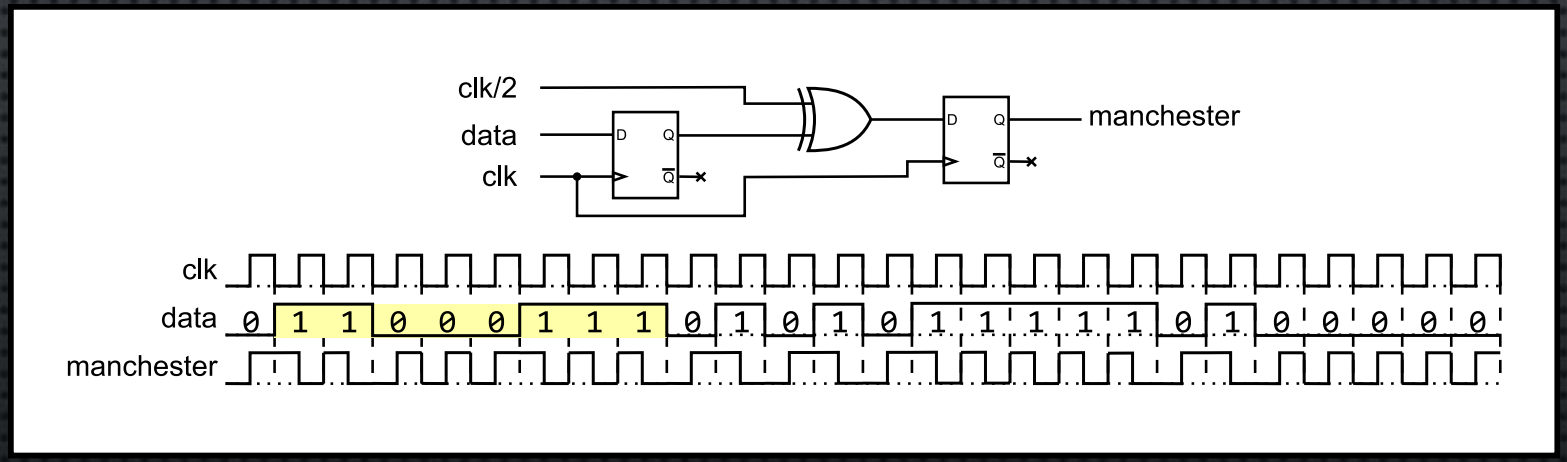
Since the timing (clock) information is carried together with the data, it is considered a synchronous transmission, even though it does not use a separate line for the clock as other types of synchronous interfaces do.

MANCHESTER ENCODER (1)

To build a Manchester Encoder in its simplest form, it would suffice to XOR the clock (clk) with the data (Stallings convention, IEEE 802.x standard).

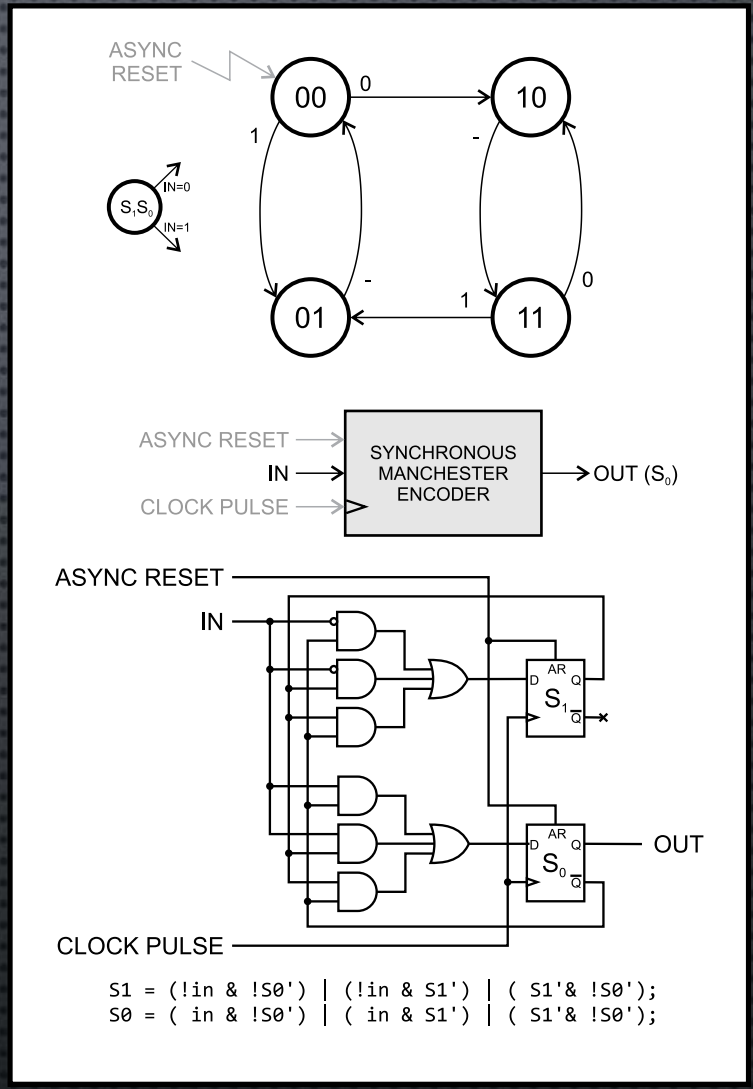
To improve the encoder's reliability, a synchronizer (flip-flop) was added at the data input.

Although it adds delay to the encoder's output signal, a second flip-flop was added in order to eliminate any static hazards generated by the XOR gate. Indeed, depending on the technology and architecture of the XOR gate's implementation, since the function consists of two disjoint minterms, there is a probability that static hazards may be generated.



MANCHESTER ENCODER (2)

- An alternative structure for the Manchester encoder uses a state machine to generate the signal.
- The advantage of this architecture is that only one clock is needed for its operation and that the probability of static hazards at the output is completely eliminated.
- On the downside, it uses more parts to implement the function.



MANCHESTER DECODER

- Decoding a Manchester signal involves recovering the timing (clock) together with the actual information (the data).
- In practice, for signals coming from very high-speed channels, clock recovery is done by special circuits (PLL, Phase Locked Loop) able to lock onto a frequency or a multiple of it. After a small number of signal cycles, such devices generate a clock perfectly synchronized with the one contained in the signal. This clock can be used to recover the original information from the signal itself.
- If the communication consists of a signal that is not particularly fast, alternative techniques can be used that do not require analog parts such as PLLs. Architectures of this kind are based on detecting changes in the signal together with delayed sampling, both obtainable with logic circuits. In this presentation we will use this second approach.

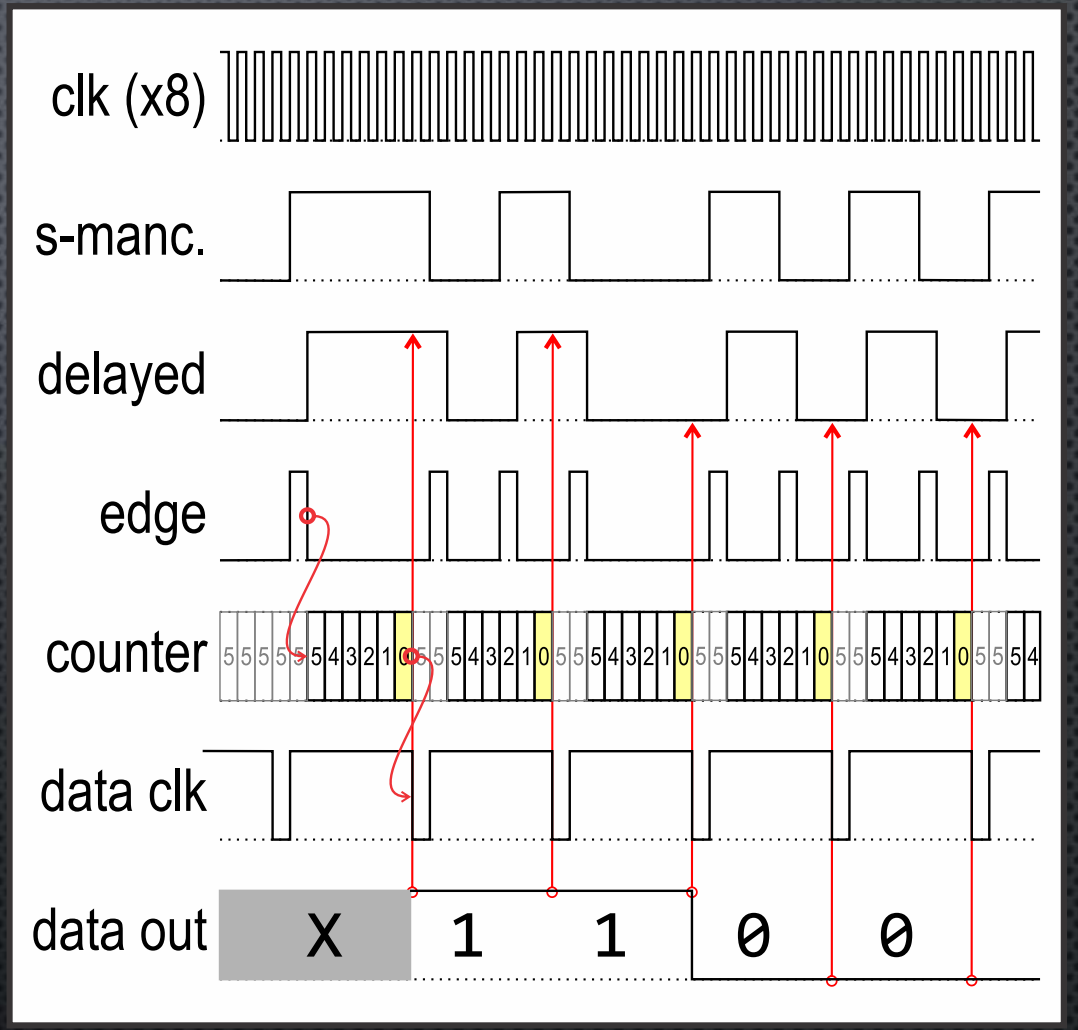
MANCHESTER DECODER

The decoder extracts the clock (data clock) and the data (data out) from the signal (s-manc).

By XORing the signal (s-manc) with a delayed copy of itself (delayed) we obtain the *edge* signal.

The *edge* signal feeds an input of a state machine (counter/sequencer). If the latter is waiting for a stimulus and if *edge* is one, the state machine, in our case starting from 5, triggers a countdown that, on reaching zero, determines the moment at which the decoder samples the value of the data contained in the input signal (s-manc).

During the count, the *edge* input is ignored.



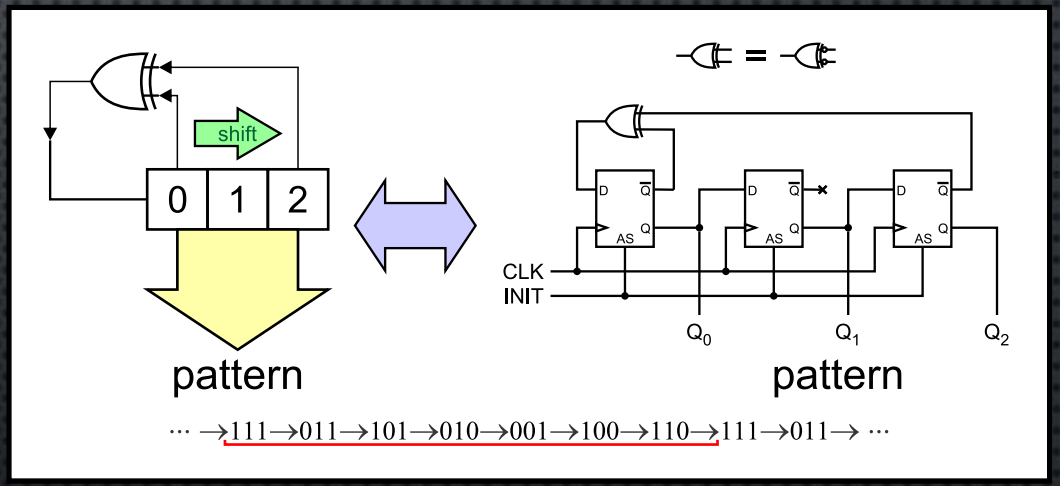
HDL IN PRACTICE: (A USEFUL INTERLUDE) LFSRS

LFSR (LINEAR FEEDBACK SHIFT REGISTER)

LFSR: it is a sort of *Ouroboros* (the snake that eats its own tail) of digital electronics, in which the output of a Shift Register is manipulated and fed back into its input so that the function cycles endlessly through a sequence of all-different numbers (expected pattern).

LFSRs are very fast, simple to build and very useful for a wide variety of applications.

One of the most common forms of LFSR is made of a simple Shift Register with feedback starting from two or more points (called *taps*) taken along the chain of registers. Alongside, an example of a three-bit LFSR with two taps.



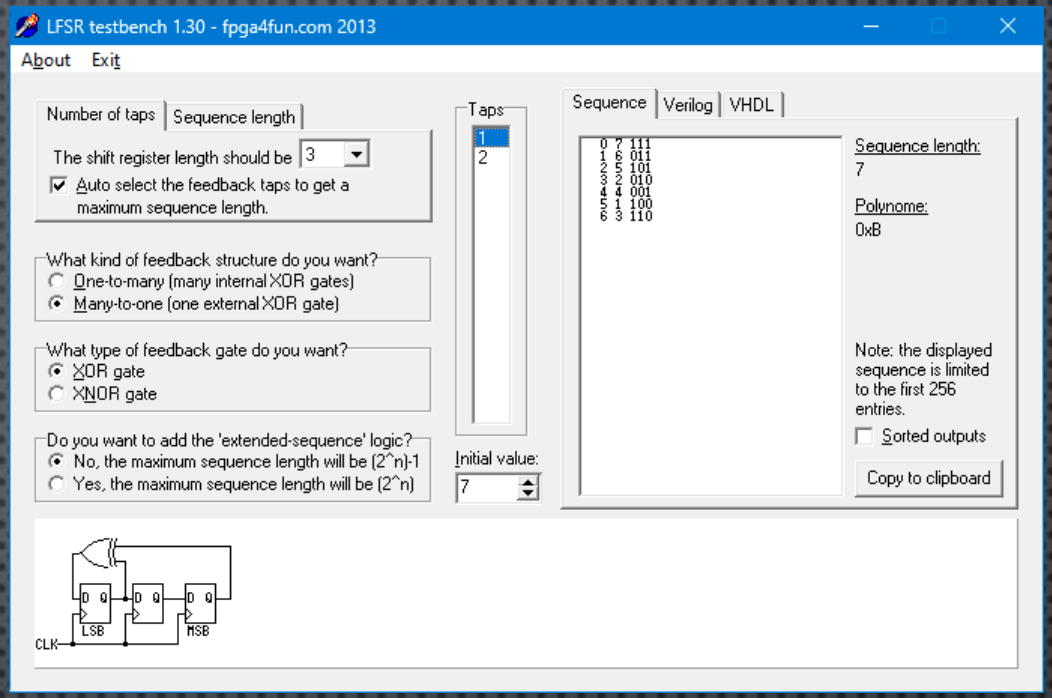
LFSR TESTBENCH

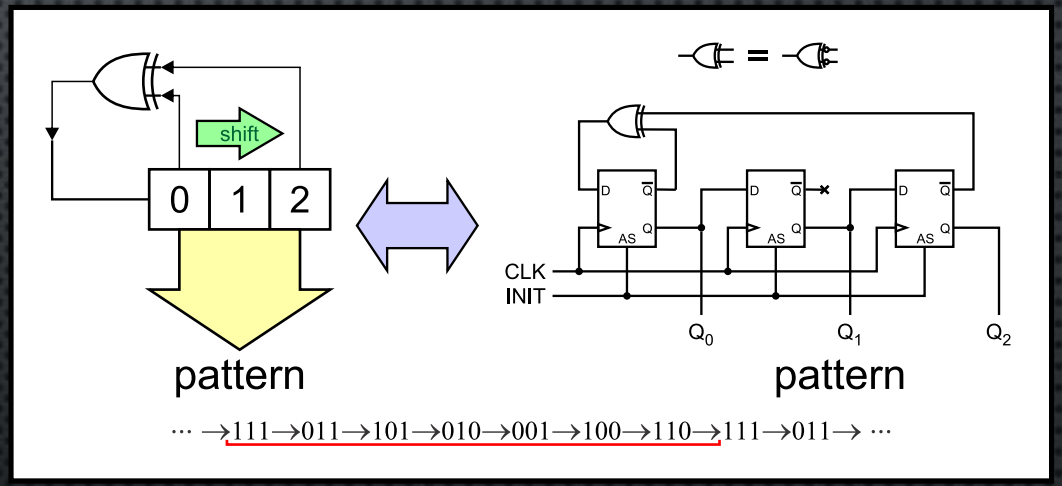
To synthesize an LFSR you can use an *open-source* tool called LFSR testbench, downloadable at this URL:

https://www.fpga4fun.com/files/LFSRTestbench.zip

Normally, in the simplest configuration, the number of codes produced (sequence length) is less than 2^n where n is the so-called "length" of the LFSR in bits.
By adding a few gates it is possible to have LFSR generators with a sequence length of 2^n.

This software lets you specify various LFSR parameters and provides the result in the form of a circuit diagram, the produced sequence, Verilog code or VHDL code.

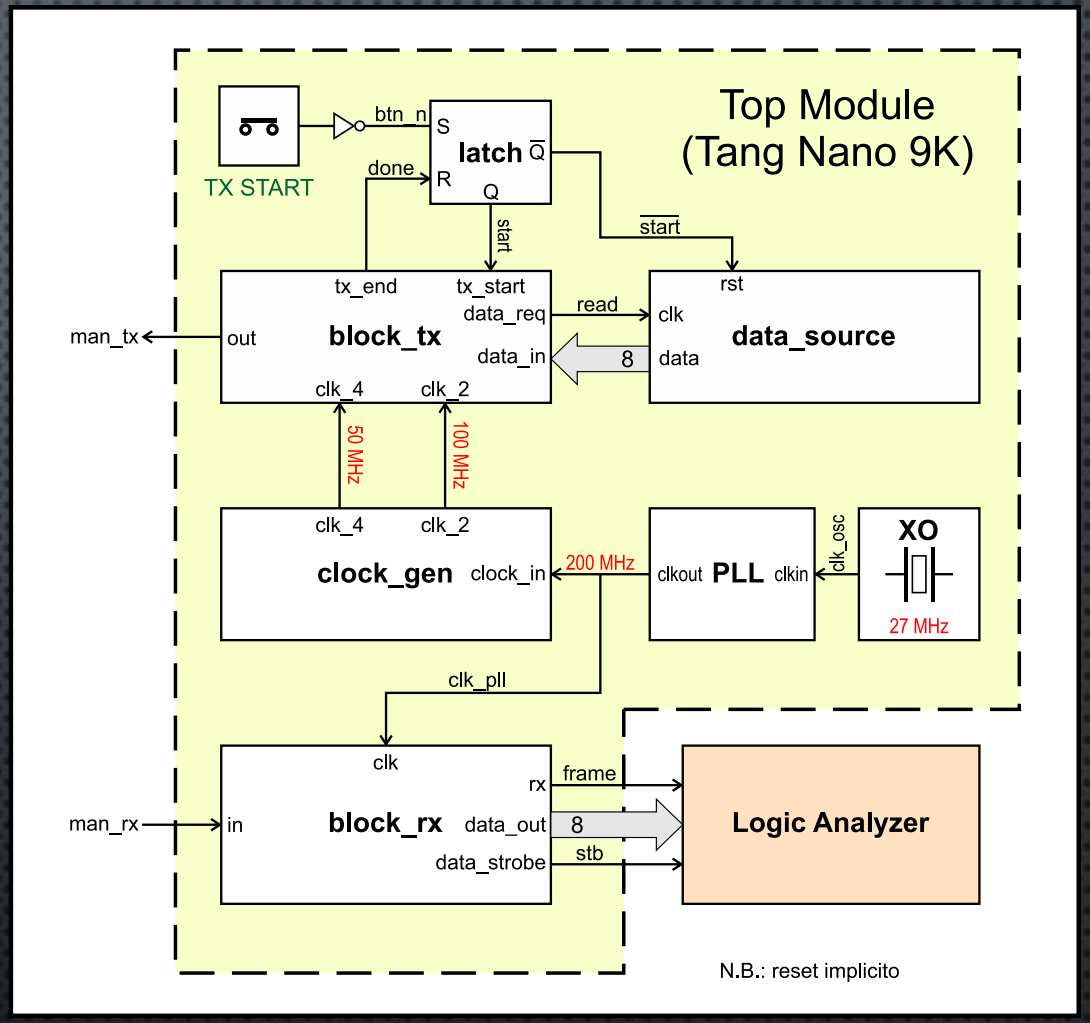




HDL IN PRACTICE: EXPERIMENT WITH MANCHESTER ENCODING

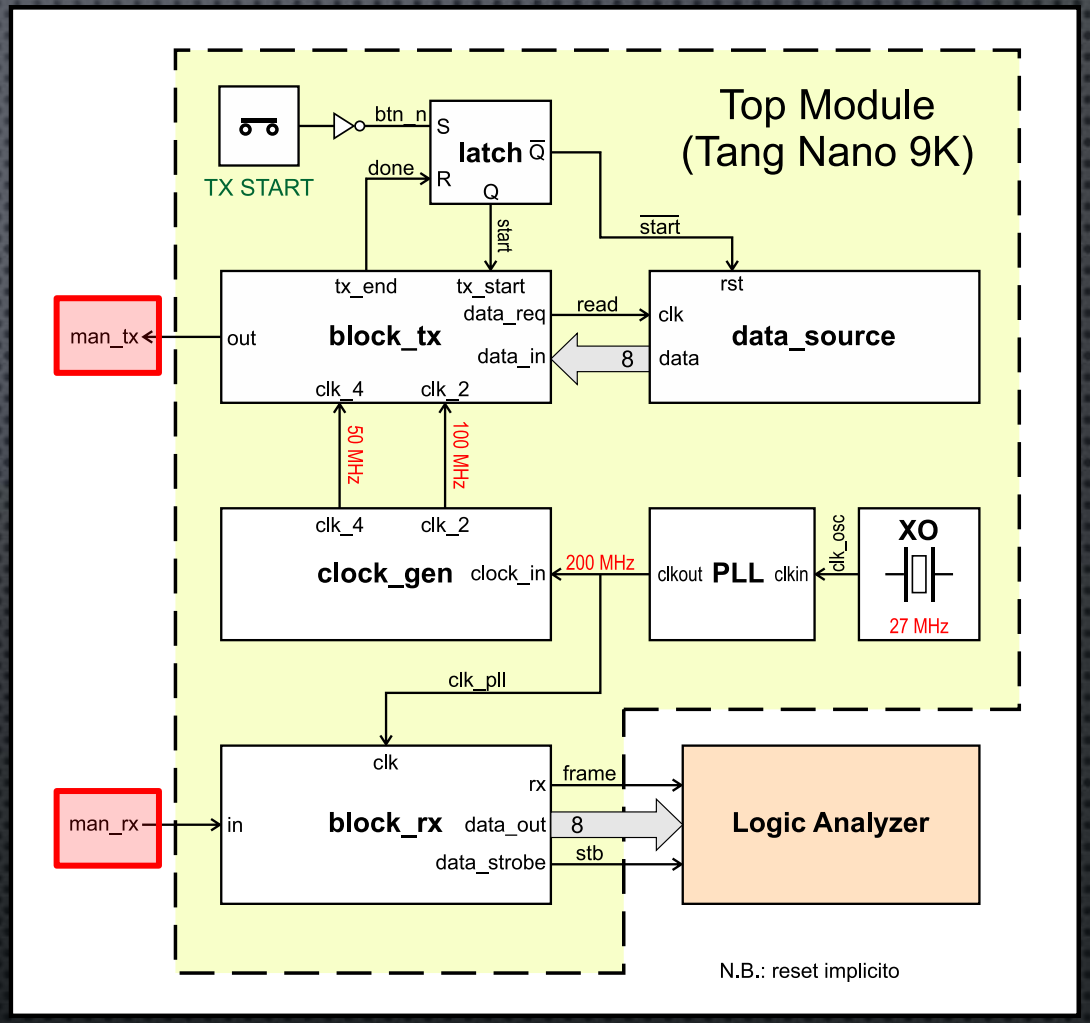
EXAMPLE: A MANCHESTER TRANSCEIVER (1)

- Our experiment, whose structure is shown alongside, is built entirely on a SiPeed Tang Nano 9K.



EXAMPLE: A MANCHESTER TRANSCEIVER (2)

- Our experiment, whose structure is shown alongside, is built entirely on a SiPeed Tang Nano 9K.
- We can observe the input and output of the Manchester signals (man_rx and man_tx respectively).



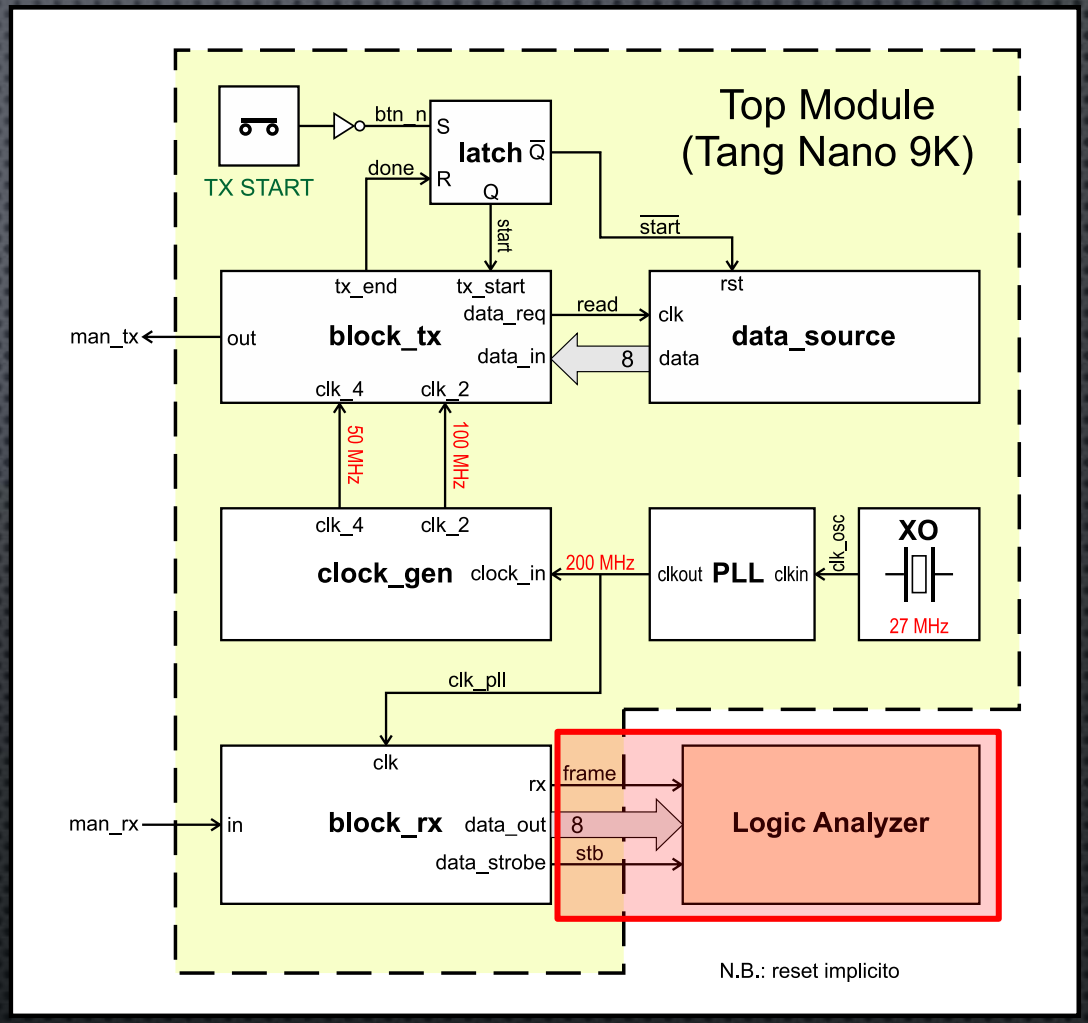
EXAMPLE: A MANCHESTER TRANSCEIVER (3)

Our experiment, whose Top Module structure is shown alongside, is built entirely on a SiPeed Tang Nano 9K.

We can observe the input and output of the Manchester signals (man_rx and man_tx respectively).

The receiver's output is presented as a group of parallel bits (data_out), together with two synchronization signals (frame and stb).

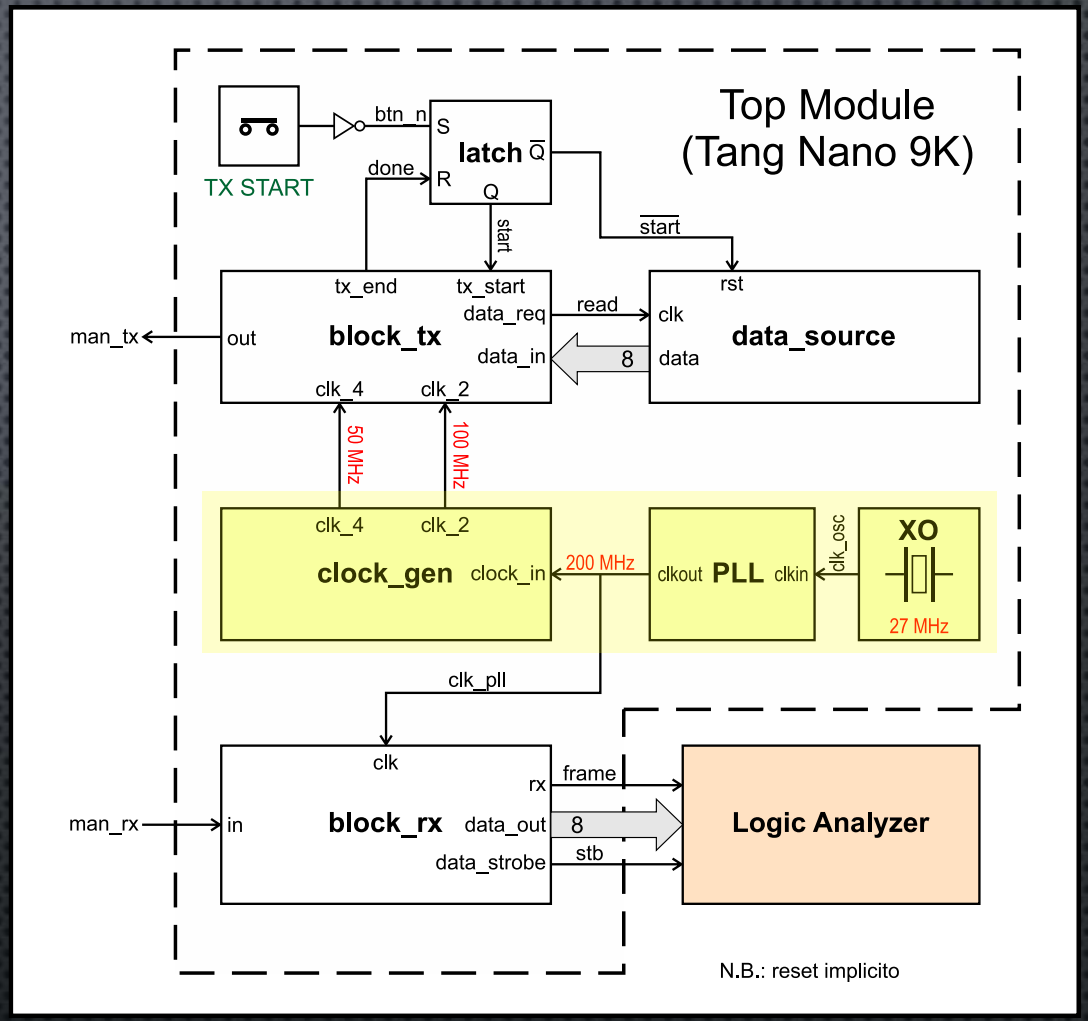
To be able to appreciate how the receiver works, the output signals will be recorded by a Logic Analyzer, which will let us observe the result.



EXAMPLE: A MANCHESTER TRANSCEIVER (4)

Inside the *top module* there are three distinct sections: clock generation, transmit section and receive section.

The first concerns the clock to be supplied to the other two sections, and is derived from the 27 MHz XO on the Tang Nano 9K. The XO output enters the FPGA and goes through a PLL: a Hard Block (or Hard IP) that multiplies the XO frequency by a fractional factor and brings it to about 200 MHz. The 200 MHz clock enters a second Hard Block that divides it by two and then by four, thus obtaining two further cadences, of 100 MHz and 50 MHz respectively, which will serve the receive and transmit sections we will see shortly.



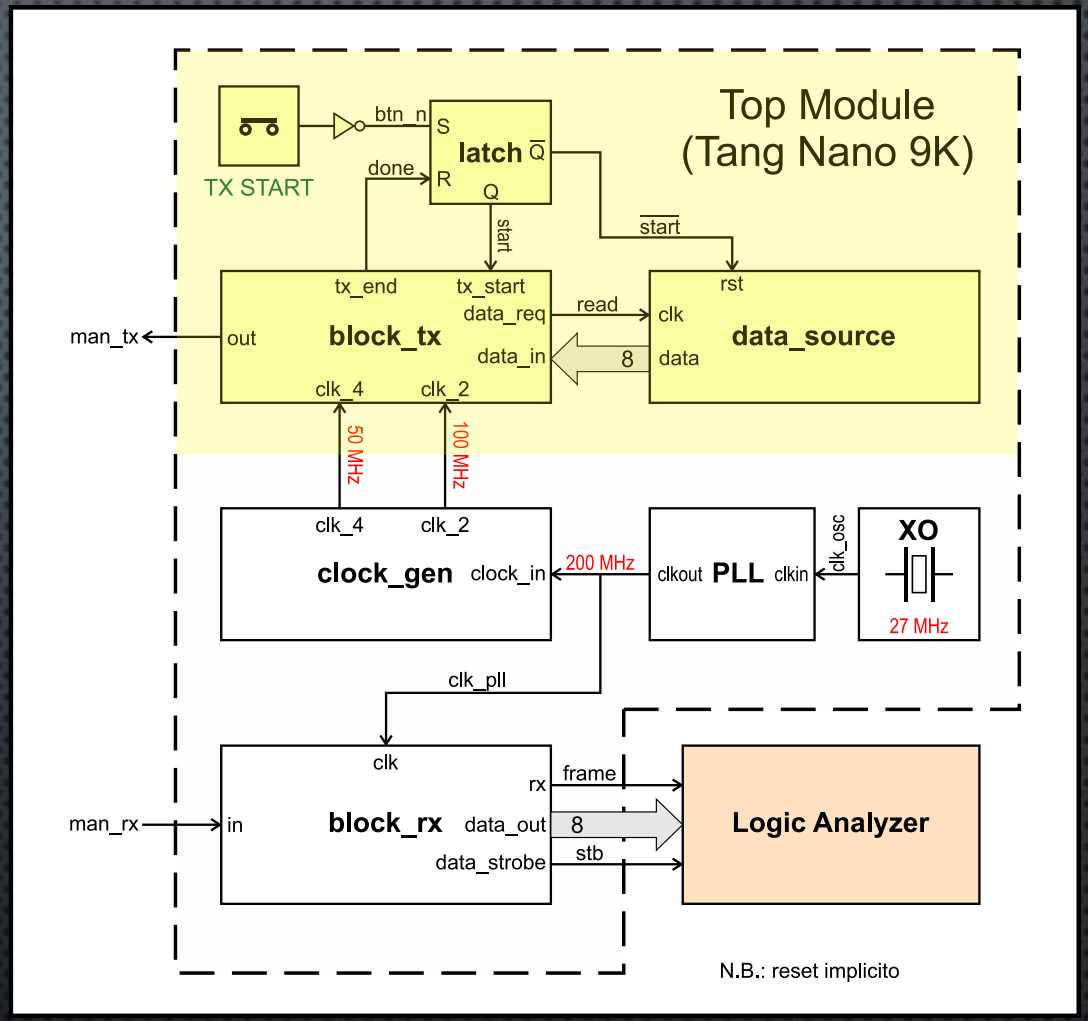
EXAMPLE: A MANCHESTER TRANSCEIVER (5)

The transmit section is made up of four parts. The transmit block (block_tx) contains the Manchester modulator as seen earlier. It generates the output signal (man_tx) and uses two clocks (clk_4 and clk_2).

A source (obtained from an LFSR) produces the parallel data (data) that block_tx will transmit serially. The LFSR will produce a new code whenever block_tx requests it with a positive edge through the net *read* that connects the two modules.

One of the two buttons on the Tang Nano 9K is connected to a latch through which a transmission request (tx_start) is triggered for a 255-byte packet (generated by the LFSR). The data packet (payload) is preceded by a two-byte preamble (0xAA55) that will be used to synchronize the receiver. In all, the frame will be 257 bytes long.

When block_tx has finished sending the data, it generates a pulse (tx_end) that clears the latch and resets data_source through the net *!start*.

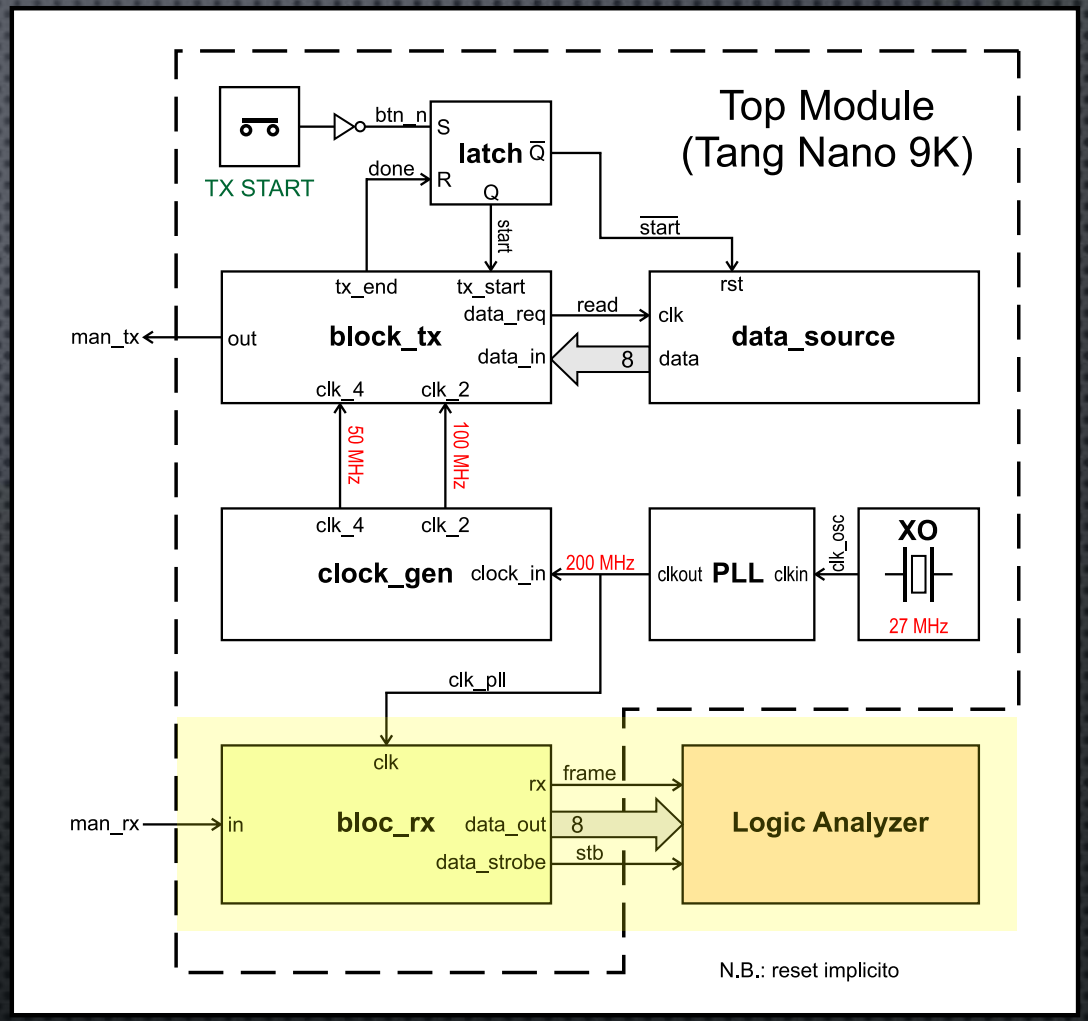


EXAMPLE: A MANCHESTER TRANSCEIVER (6)

The input signal (man_rx) reaches block_rx: this is the module in charge of decoding the Manchester protocol. It receives the faster clock (clk_pll) and produces two scalar signals (frame and stb) and an eight-bit vector (data_out).

The task of block_rx is to synchronize with the transmitter, extract the actual data (the payload) and present it consecutively as bytes at the output.

A Logic State Analyzer is connected to the output of block_rx, allowing us to see what was received.



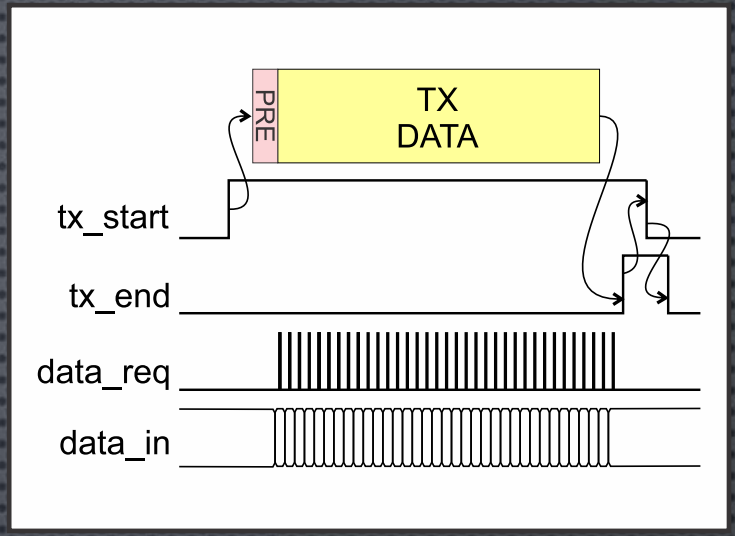
BLOCK_TX MODULE

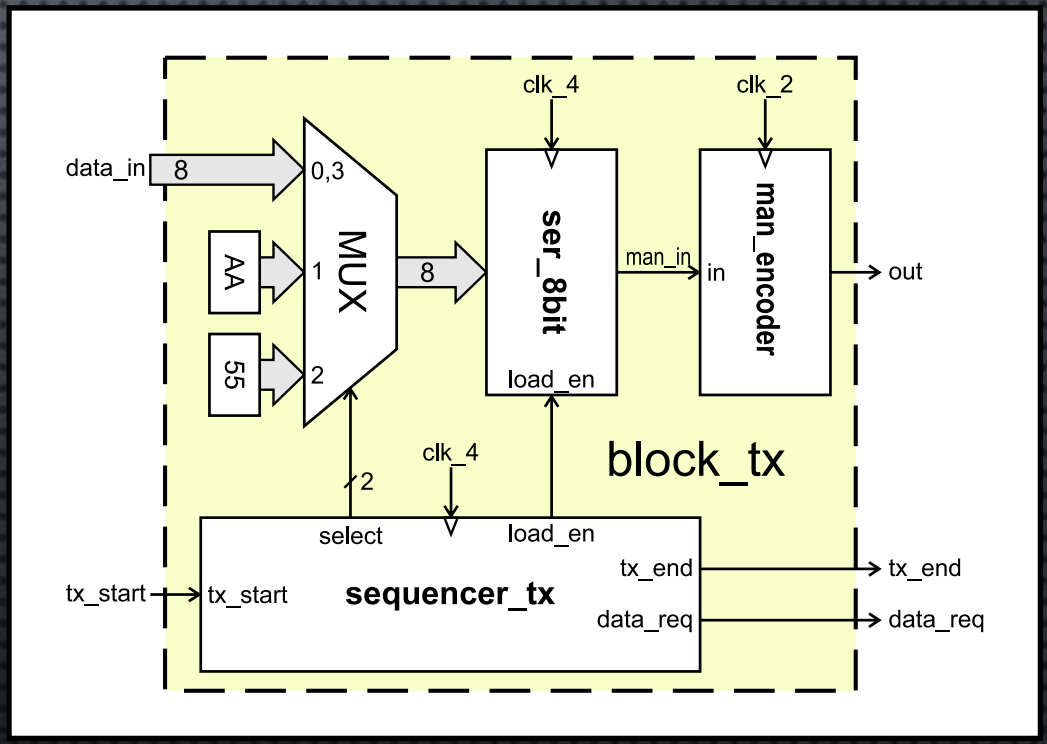
The block_tx module uses three submodules: ser_8bit, man_encode and sequencer_tx. The ser8_bit module is a serializer (PISO) as already seen earlier. The man_encode module, already explained, generates the Manchester signal.

The block_tx module receives the data (the payload) in parallel format from an eight-bit vector (data_in). The data enter a multiplexer (MUX) coded directly in the module. This multiplexer also selects, one at a time, the two bytes used for the synchronization preamble sequence.

The sequencer_tx module is in charge of generating the signals that orchestrate the operation of the whole block_tx module. When the sequencer_tx module receives a high level on the tx_start input, it first instructs the multiplexer to select the first byte of the synchronization preamble (0xAA).
At the same time it enables the serializer (ser_8bit), which sends the bits one by one to the Manchester encoder (man_encoder), which generates the output signal. Next, sequencer_tx selects the second synchronization byte (0x55) and sends it. After that, it begins sending the whole payload that is requested, byte by byte, generating useful edges through the data_req signal.

Finally, sequencer_tx will signal the end of the payload transmission with tx_end. At this point sequencer_tx will wait for the start_tx input to return to zero.
When start_tx is deasserted, tx_end will also return to zero.





BLOCK_RX MODULE

The block_rx module contains 5 submodules. The input signal (in) meets a first synchronization block (sync_meta) that produces a signal (in_d) synchronized with the clock (clk).

The in_d signal arrives at the man_decoder module, which is responsible for extracting the information bits (man_out) and the clock related to them (man_clk).

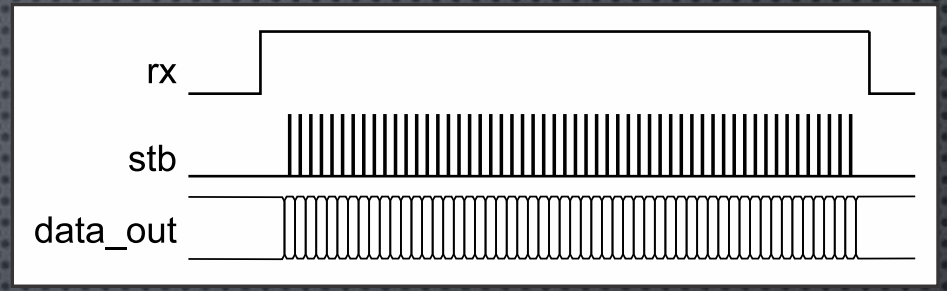
The man_out signal is fed to the des_n_bit module, which is a 16-bit deserializer (SIPO). The deserialized bits are propagated inside the module through a 16-bit bus (par_our) that reaches sequencer_rx, i.e. the state machine that controls the operation of the whole module.

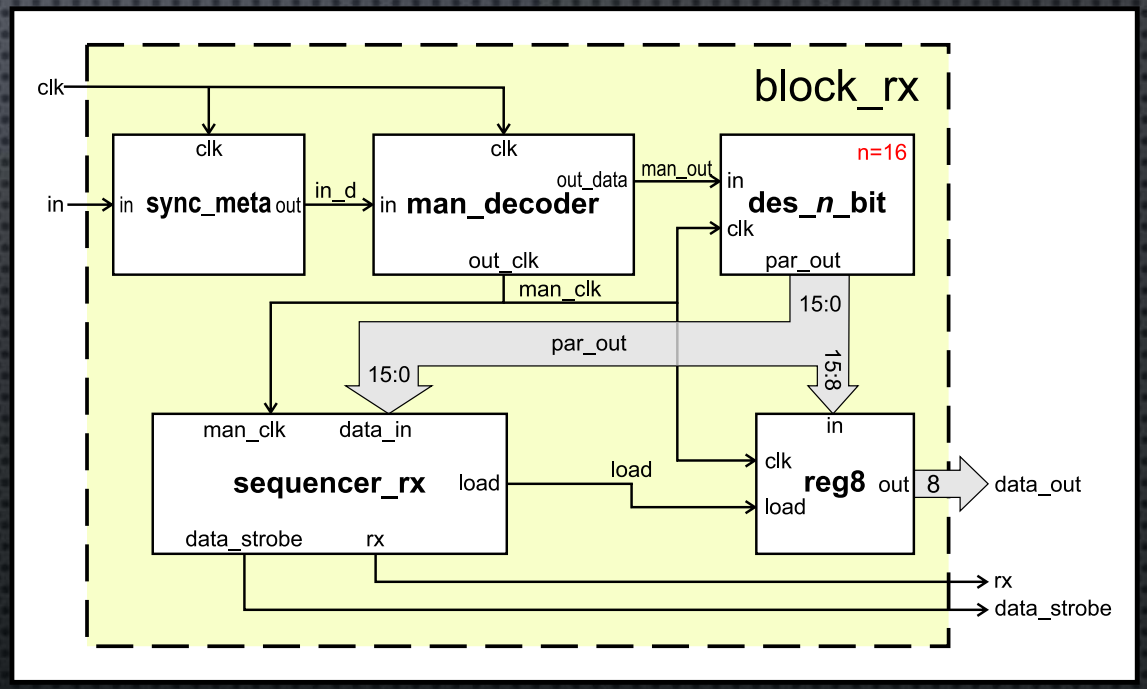
When sequencer_tx reads the 0xAA55 pattern (the preamble), it recognizes the following bits as payload. The rx signal is asserted to indicate that the block of bits carrying the payload is arriving.

Every 8 bits of payload, sequencer_tx produces a pulse that instructs reg8 (an 8-bit register) to sample the eight most significant bits of par_out (par_out[15:8]).

Next, sequencer_tx generates the data_strobe pulse that notifies the recipient of the data, in our case the Logic Analyzer, that a payload byte is ready to be read.

At the end of the payload, sequencer_rx deasserts the rx signal, indicating that the whole frame has ended.





Q&A

- Speakers:

Emanuel Conti (e.conti@embedded.sm)

Giuliano Cardinali (giulicard@gmail.com)

Source code download:

https://tinyurl.com/HDLParte2Code

BIBLIOGRAPHY

- Digital Design and Computer Architecture 2nd Edition
  David Harris, Sarah Harris - ISBN978-0123944245
- Reti logiche e calcolatori
  F. Luccio, L. Pagli – ISBN: 978-8833954870
- FPGAs: Instant Access
  Clive Maxfield - ISBN: 978-0750689748
- Bebop to the Boolean Boogie: An Unconventional Guide to Electronics
  Clive Maxfield - ISBN: 978-1856175074
- Logic Circuit Design: Selected Topics and Methods, Second Ed.
  Shimon P. Vingron - ISBN: 978-3031406720
- The Verilog® Hardware Description Language, 5th Ed.
  Donald E. Thomas, Philip R. Moorby - ISBN: 978-1402070896
- Digital design : with an introduction to the verilog hdl
  M. Morris Mano, Michael D. Ciletti.—5th ed. - ISBN-13: 978-0132774208
- Digital Design Principles and Practices
  John F. Wakerly - ISBN: 978-0134460093